Classes and Instances

(https://github.com/learn-co-curriculum/python-p3-classes-and-instances) (https://github.com/learn-co-curriculum/python-p3-classes-and-instances/issues/new)

Learning Goals

- Describe a Python class and how it creates objects.
- Describe a Python instance.
- Create an instance of a class.

Key Vocab

- **Class**: a bundle of data and functionality. Can be copied and modified to accomplish a wide variety of programming tasks.
- **Initialize**: create a working copy of a class using its `__init__()` method.
- **Instance**: one specific working copy of a class. It is created when a class's `__init__()` method is called.
- **Object**: the more common name for an instance. The two can usually be used interchangeably.
- **Function**: a series of steps that create, transform, and move data.
- **Method**: a function that is defined inside of a class.

Introduction

Let's say we are building a dog walking app. Our app's users might be dog walkers and dog owners and they can use the app to manage the dog walks. Such an app would need to store information about a potentially large number of dogs.

Our program needs to have a way to bundle up and operate on all the information about a particular dog. And, our program needs to be able to do this again and again. And, once more, we'll need our program to be able to create *new* bundles of information regarding individual dogs every

time a new dog is added to the app.

How can we tell our Python program to deal with these dogs? Well, we can write a `Dog` class that produces individual dog objects, each of which contains all the information and behaviors of an individual dog.

Defining a Class

Think of a class like a blueprint that defines how to build an object. The `Dog` class is different from an individual dog just as the blueprints that show how to build a house are not the actual house. A Python class both contains the instructions for creating new objects and has the ability to create those objects. A constructor is a special function used to create an instance of a class. Calling the `Dog` class constructor is like getting a brand new dog object from an assembly line which produces a series of similar dog objects based on the same `Dog` template.

Here's what our `Dog` class would look like:

```
class Dog:
  # some code to describe a dog

# new code goes here
```

The `Dog` class is defined with the `class` keyword, followed by the class name and closed with a deindentation to the level where we defined the `Dog` class. The body of this class is the code block between our first indentation and our final deindentation.

Class names begin with capital letters because they are stored in Python constants. If your class name contains two words, the name should be in UpperCamelCase (also called *PascalCase* if you're feeling fancy), like this:

```
class MyClass
  # some code all about your awesome class

# new code goes here
```

With this code alone, we can now make new dogs!

Creating Instances of Classes

Open up the Python shell, or create a new Python file to code along, and enter the following code:

```
# Remember that Python requires some indented code in each code block.
# Instead of empty blocks, we use the "pass" keyword to do nothing.

class Dog:
    pass

fido = Dog()
fido
# <__main__.Dog object at 0x1049a87f0>
```

In the code sample above, once we've defined our `Dog` class with the `class` keyword, we immediately can bring to life new individual dogs, the variable `fido` which points to a new **instance** of a dog.

We **instantiate** an instance of the `Dog` class by calling the constructor function `Dog()`. This creates our **instance** of the `Dog` class, which we assign to the variable `fido`. An instance of a class is also referred to as an object.

Instantiate means bringing a new object to life, a new individual, like a particular dog, like Snoopy or Lassie or Rover. Each particular dog is an individual that was **instantiated** when we called the constructor function `Dog()` to birth it into our world of programming.

We call these individuals, each specific dog or version of our class, **instances**. An **instance** is a single occurrence of an **object. Instances** refer to the individual objects produced from the class.

```
class Dog:
    pass

fido = Dog()
fido
# <__main__.Dog object at 0x1049a87f0>
```

```
snoopy = Dog()
snoopy
# <__main__.Dog object at 0x104971d90>
```

`snoopy` and `fido` are two different variables pointing at separate instances of the `Dog` class.

Different Instances are Different Objects

Let's make three dogs:

```
class Dog:
    pass

fido = Dog()
fido
# <__main__.Dog object at 0x1049a87f0>

snoopy = Dog()
snoopy
# <__main__.Dog object at 0x104971d90>

lassie = Dog()
lassie
# <__main__.Dog object at 0x10498c040>
```

Notice that every time you make an instance of a class, Python tells you that the return value is something that looks like `# <__main__.Dog object at 0x1049a87f0>`. This is the default way that Python communicates to you that you are dealing with an instance of a particular class. The `__main__` tells you that the object is accessible from a global scope in the current module (file or shell) that you're working in. `0x1049a87f0` describes the instance's location in memory.

Each of these instances is totally unique, even though they are all born from `Dog`.

```
class Dog:
    pass

fido = Dog()
fido
# <__main__.Dog object at 0x1049a87f0>

snoopy = Dog()
snoopy
# <__main__.Dog object at 0x104971d90>

snoopy == fido
# False
```

Classes are the blueprints that define the behavior and information our objects will contain. They let us manufacture and instantiate new instances.

Conclusion

In summary: to create a new class definition, use the `class` keyword. A class is like a template, or a blueprint, for creating objects with similar characteristics.

To use the class to create individual objects, call the class with closed parentheses as you would a function or method. This will **instantiate** (create a new **instance** of) an object from the class. Each instance created will be a unique object in memory.

Resources

- **Classes - Python (https://docs.python.org/3/tutorial/classes.html)**
- **Python Classes and Objects (https://www.geeksforgeeks.org/python-classes-and-objects/)**